

# MCP-**Universe**

## Benchmarking LLMs with Real-World MCP Servers

실제 MCP 서버에 연결된 대규모 벤치마크를 통해  
에이전트의 실제 도구 사용 성능을 평가하는 포괄적 프레임워크

231

태스크

11

MCP 서버

133

도구

84

Evaluator

● 실제 서버 연결    ● 다중 도메인    ● 실시간 평가



발표자: 김강민

GitHub: [SalesforceAIResearch/MCP-Universe](#)



Agenda

## 목차

총 40장

예상 시간: 30-45분



### 1. 도입

01

에이전트와 도구 사용

개념

예시



### 2. 배경 논문

02

AgentBench, StableToolBench

리뷰

분석



### 3. MCP-Universal

03

본문: 벤치마크 설계

구조

평가



### 4. Key Findings

04

핵심 발견

분석

결과



### 5. 리더보드

05

성능 비교

순위

메트릭



### 6. 최신 동향

06

2025-2026 동향

트렌드

연구



### 7. Q&A

07

예상 질문

FAQ

답변



### 8. 참고문헌

08

링크/자료

논문

GitHub



### 9. 결론

09

정리 및 요약

요약

인사이트



Part 1

# 에이전트란 무엇인가

목표

에이전트의 핵심 개념 이해

## LLM 에이전트의 정의

LLM 에이전트는 **관찰** → **추론** → **행동** → **피드백**을 반복하는 시스템입니다. 단순 질의응답이 아닌 **다중 스텝 상호작용**을 통해 환경 변화와 불확실성에 적응합니다.

다중 스텝

상호작용

적응

## 핵심 특징

● 환경 관찰

● 행동 실행

● 추론 능력

● 피드백 학습

## 에이전트 순환 구조

4단계 반복



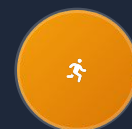
관찰

Observation



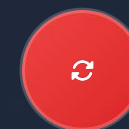
추론

Reasoning



행동

Action



피드백

Feedback



**에이전트의 핵심:** 단순한 질의응답(QA) 시스템이 아닌, 주어진 환경과 **상호작용**하며 동적으로 목표를 달성해 나가는 시스템입니다.



Part 1

## 두바이-아부다비 중간 지점 레스토랑 찾기

예시

다중 스텝 도구 사용

### 문제 상황

"두바이 Sheraton Jumeirah Beach Resort에 살고, 동료는 아부다비 Oryx Hotel에 산다.  
두 호텔에서 운전 시간이 비슷한 레스토랑(평점 4.2 이상)을 찾아라."

위치 조회

중간 지점 계산

레스토랑 검색

### 핵심 포인트

이 태스크는 **최소 5~8 스텝**의 다중 도구 호출이 필요합니다. 한 번의 API 호출로 끝나는 것이 아니라, **다중 스텝 과정**을 거칩니다.

### 실제 도구 사용 과정

6단계 실행

1 **Google Maps에서 좌표 검색**  
Sheraton Dubai, Oryx Abu Dhabi 위치 확인



2 **중간 지점 계산**  
두 호텔 사이 중간 좌표 계산



3 **주변 레스토랑 검색**  
중간 지점 주변 레스토랑 찾기



4 **평점 필터링**  
4.2 이상 평점 레스토랑 선별



5 **운전 시간 비교**  
각 레스토랑까지의 운전 시간 계산



6 **결과 보고**  
최종 선택 및 정보 제공



**i** 이 태스크는 **최소 5~8 스텝**의 다중 도구 호출이 필요합니다.

성공률: 26.67% | 평균 스텝: 6.84



Part 1

## 도구 사용의 3단계

프로세스

도구 선택 → 인자 구성 → 결과 해석

### 1 도구 선택

Tool Selection



"Google Maps를 쓸까, Fetch를 쓸까?"  
→ 올바른 **MCP** 서버/도구 선택



API 상태 확인, 기능 비교

### 2 인자 구성

Parameter Configuration



"출발지=Sheraton Dubai, 도착지=Oryx Abu Dhabi"  
→ **파라미터 정확히 채우기**



인자 유효성 검증

### 3 결과 해석

Result Interpretation



"경로 1시간 40분 → 중간 지점 계산"  
→ **다음 행동 계획**



결과 분석, 다음 스텝

### 3단계 프로세스 흐름

완료 시 성공, 하나라도 실패 시 전체 실패

도구 선택

MCP 서버 선택



인자 구성

파라미터 설정



결과 해석

다음 행동 결정



이 세 단계 중 **하나만 실패하더라도** 전체 태스크가 실패합니다.



Part 1

## 도구 사용 벤치마크가 어려운 4가지 이유

핵심 포인트  
재현성·공정성 저해 요인



### 1. API 상태 변화 재현성 ↓

어제 되던 API가 오늘 인증 만료로 죽음. API 상태가 시간에 따라 변화하여 **재현성**을 저해합니다.

🕒 시간 민감성



### 2. 실행 비결정성 결과 불일치

동일 입력이지만 서버 상태에 따라 다른 결과. **실행 비결정성**으로 인해 결과가 달라집니다.

🖨 서버 상태 의존



### 3. LLM-judge 랜덤성 채점 불안정

ChatGPT로 채점하면 같은 답안도 때마다 다른 점수. **LLM-judge**의 랜덤성 문제.

📊 평가 일관성



### 4. 시간 민감 정답 정답 변화

"현재 AAPL 추가"의 정답은 1분 뒤에 달라짐. **시간 민감** 정답으로 인해 평가가 어려워집니다.

📈 실시간 데이터

❗ 이러한 복합적인 문제들이 객관적이고 재현 가능한 벤치마크 설계를 매우 어렵게 만듭니다.



## AgentBench 개요 (ICLR 2024)

AgentBench는 "LLM을 에이전트로 평가하려면 무엇을 측정해야 하는가?"라는 질문에 답하기 위해 개발된 최초의 **다중 환경 에이전트 벤치마크**입니다. 8개 환경(OS, DB, KG, Digital Card Game, LTP, House-Holding, Web Shopping, Web Browsing)에서 동일 인터페이스로 29개 모델을 평가했습니다.

8

환경

29

모델

4

실패 유형

다중 환경

표준화 평가

실제 데이터

## 핵심 발견

- 상용 모델(GPT-4)과 오픈소스( $\leq 70B$ ) 사이에 큰 성능 격차
- 실패 유형: IF, IA, TLE, CLE
- "코드 특화 = 에이전트 특화" 등식은 성립하지 않음

## 환경 구성



OS



DB



KG



Web

**의의:** 최초의 다중 환경 에이전트 벤치마크

✓ 8개 환경, 29개 모델 평가



Part 2

# AgentBench 주요 발견

ICLR 2024

8개 환경, 29개 모델 평가

## 상용 vs 오픈소스 성능 격차

2.32

API-based 모델

평균 점수

0.51

OSS 모델 ( $\leq 70B$ )

평균 점수

4.5x

성능 격차

압도적 차이

상용 API 모델과 오픈소스 (OSS) 사이에 큰 성능 격차가 존재합니다.

## 주요 실패 유형

IF (형식 오류)

IA (행동 오류)

TLE (턴 초과)

CLE (컨텍스트 한계)

## 핵심 발견사항

8개 환경 평가 결과

</> OS/DB/KG

시스템/데이터베이스/지식그래프

+ 게임/웹

카드게임, 웹쇼핑, 브라우징

추론

LTP, House-Holding

자동화

WebShop, Web Browsing



"코드 특화 = 에이전트 특화" 등식은 성립하지 않음

29개 모델 평가 | 8개 환경





Part 2

## StableToolBench 요약

문제 제기  
실제 REST API의 불안정성

### ⚠️ 핵심 문제

ToolBench(16,000개 이상의 실제 REST API)의 API를 스캔했더니 **성공 호출 비율이 44.4%**에 불과. 절반 이상이 죽어 있었다.

API 불안정

재현성 문제

### ✅ 해결책

가상 **API** 서버와 **SoPR/SoWR**로 안정화. 캐시 + 시뮬레이터 + 호출 규칙으로 API 불안정 흡수.

가상화

SoPR/SoWR

### 두 축의 안정화

재현성 vs 현실성

**44.4%**

실제 API 성공률

**100%**

가상 API 성공률

**+55.6%**

안정성 개선

**16,000+**

실제 REST API

**44.4%**

성공 호출

**55.6%**

실패 호출

**100%**

가상화 후



# StableToolBench: 해결책과 트레이드오프

재현성 vs 현실성  
44.4% 성공 호출률

## 문제 제기

실제 REST API의 불안정성 - ToolBench(16,000+개 실제 REST API)를 스캔했더니 성공 호출 비율이 44.4%에 불과. 절반 이상이 죽어 있었다.



## 핵심 해결책

### 가상 API 서버

캐시 + 시뮬레이터 + 호출 규칙

### SoPR/SoWR

Solvable task만 평가

## 트레이드오프 분석

재현성 vs 현실성



### 장점

- 재현성 확보
- 공정 비교 가능
- 안정적 평가



### 단점

- 현실 오류 반영 약화
- 인증/레이턴시 무시
- 예상 못한 응답 놓침



### 결론

안정성을 올리면  
현실성이 내려간다  
→ MCP-Universe의 출발점

StableToolBench는 재현성을 위해 현실성을 희생하는 트레이드오프를 보여줍니다.

논문: StableToolBench (ACL 2024)



Part 2

# MCP(Model Context Protocol) 소개

목표

MCP의 구조, 기능, 효과 이해

## MCP 구조 JSON-RPC 2.0 기반



Host

LLM 앱

JSON-RPC



Client

JSON-RPC 2.0

Tools/Resources



Server

외부 도구/데이터

Tools

Resources

Prompts

## 핵심 기능



Tools

실행 함수



Resources

데이터 접근



Prompts

컨텍스트 제공

## MCP의 효과와 현황

2026년 4월 기준

2024

Anthropic 제안

OpenAI

Assistants → MCP

Google

관리형 MCP 지원

de facto

표준 프로토콜



표준화

모델-도구 M×N 연결



안정성

API 호출 안정화



확장성

다양한 도구 연결



Part 2

# MCP 채택과 생태계

2024년 11월  
Anthropic 제안



OpenAI

Assistants → MCP

전환 상태



예  
정

- Assistants → Responses API (MCP 지원 포함)
- Strict mode 지원
- 병렬 도구 호출



Google

관리형 MCP

지원 상태



공  
식

- 관리형 원격 MCP
- Vertex AI 통합
- 보안 인증



Anthropic

MCP 개발자

생태계



활  
성

- MCP 프로토콜 제안
- Tool Search Tool
- Agent Skills



MCP는 2026년 현재 사실상의 표준(De facto standard)으로 자리잡았습니다.

모델-도구 연결의 표준화 완성

# MCP-Universal 개요와 문제의식

실제 MCP 서버에 연결된 대규모 벤치마크 프레임워크

## MCP-Universal의 핵심 특징

실세계 MCP 서버에 직접 연결된 벤치마크로, 11개의 실제 운영 서버와 133개의 도구를 포함

231

태스크

11

MCP 서버

133

도구

84

Evaluator

## 해결하고자 하는 문제

- 합성/정적 환경: 기존 벤치마크는 실제 MCP 서버와 복잡성이 다름
- 작은 도구 공간: 수십 개 도구 vs 현실의 수백 개
- 짧은 Horizon: 1-3 스텝 vs 실제 10+ 스텝 필요
- 정적 정답: 시간 민감 태스크 커버 불가

## MCP-Universal 아키텍처 및 구성



Figure 1: MCP-Universal은 실제 MCP 서버에 연결된 대규모 벤치마크로, 6개 도메인의 231개 태스크를 포함

## 핵심 목표

- 실제 도구 사용
- 장기 다중 턴
- 장문 컨텍스트
- 대규모 도구 공간
- 동적 정답

실행 기반 Evaluator를 통해 LLM-as-a-judge 대신 객관적 검증 현



## 기존 벤치마크의 4가지 한계

1

### 합성/정적 환경

Synthetic/Static Environment

기존 벤치마크는 대부분 **합성된 환경**에서 테스트됩니다. MCP-RADAR는 HumanEval, GSM8k를 MCP로 변환한 것으로, 실제 MCP 서버의 복잡성과 다릅니다.

현실성

2

### 작은 도구 공간

Small Tool Space

기존 벤치마크는 **수십 개 도구**만 사용. 현실에서는 하나의 MCP 서버만 연결해도 수십 개, 여러 서버면 수백 개의 도구가 필요합니다.

3

### 짧은 Horizon

Short Horizon

대부분의 벤치마크는 **1~3 스텝**으로 제한됩니다. 실제 복잡한 업무는 10스텝 이상의 다중 턴 상호작용이 필요합니다.

4

### 정적 정답

Static Answer

주가, 날씨, 실시간 교통 같은 태스크의 정답은 **계속 변합니다**. 고정 정답으로만 평가하면 이런 태스크를 커버하지 못합니다.



MCP-Universe는 이 4가지 한계를 모두 해결하여 **실세계 환경과 일치하는** 벤치마크를 제공하고자 설계되었습니다.



Part 3

## 벤치마크 설계 규모와 구성

규모

231개 태스크, 11개 MCP 서버

### 기본 구성

태스크

231개

231

MCP 서버

11개

11

도구

133개

133

Evaluator

84개

84

### Evaluator 구성

4

Format

32

Static

48

Dynamic

### 도메인 분포

Location

Navigation

Repository

Management

Financial

Analysis

3D

Designing

Browser

Automation

Web

Searching

총 231개 태스크, 11개 MCP 서버, 133개 도구

성공률: 43.72%



Part 3

# 6개 도메인 개요

총 태스크  
231개



✓ Active

## Location Navigation

Google Maps, Open-Meteo 활용  
좌표 계산, 경로 탐색

태스크: 45개



✓ Active

## Repository Management

GitHub MCP 활용  
레포 생성, 브랜치 관리

태스크: 38개



✓ Active

## Financial Analysis

Yahoo Finance MCP 활용  
주가 분석, MA 계산

태스크: 42개



✓ Active

## 3D Designing

Blender MCP 활용  
3D 오브젝트 생성

태스크: 28개



✓ Active

## Browser Automation

Playwright 활용  
웹 자동화, 스크래핑

태스크: 35개



✓ Active

## Web Searching

Google Search, Fetch 활용  
정보 검색, 종합

태스크: 43개

**i** 각 도메인은 실제 MCP 서버와 연결되어 실시간 데이터를 활용합니다





도메인 예시

# Location Navigation

중간 지정 레스토랑 찾기  
Google Maps + Open-Meteo

## 목표

"두바이 Sheraton Jumeirah Beach Resort에 살고, 동료는 아부다비 Oryx Hotel에 산다. **두 호텔**에서 **운전 시간**이 비슷한 **레스토랑**(**평점 4.2 이상**)을 찾아라."

위치 조회

중간 지정 계산

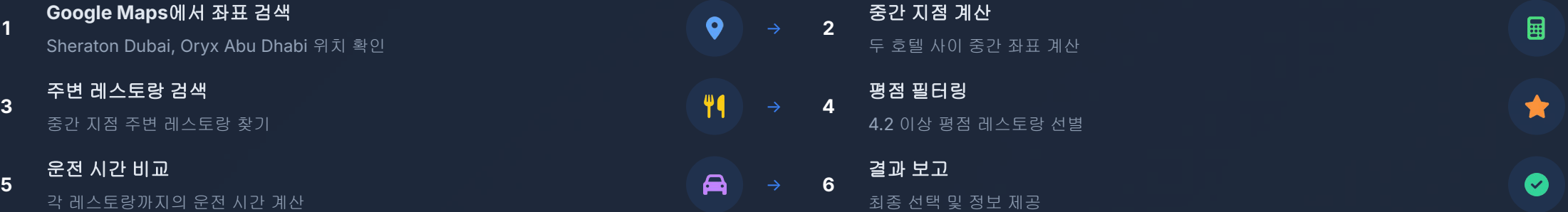
레스토랑 검색

## 평가 기준

|           |     |
|-----------|-----|
| 운전 시간 근접성 | 40% |
| 평점 필터링    | 30% |
| 형식 검증     | 30% |

## 실제 도구 사용 과정

6단계 실행



**i** 이 태스크는 **최소 5~8 스텝**의 다중 도구 호출이 필요합니다.

성공률: 26.67% | 평균 스텝: 6.84



## 목표

"GitHub에 새 레포를 만들고, README를 초기화하고, 'dev' 브랜치를 생성한 뒤, 이슈 #1을 등록하라."

레포 생성

README 초기화

브랜치 생성

이슈 등록

## 핵심 포인트

이 태스크는 실제 GitHub에 조작이 가해짐을 의미합니다. Evaluator는 GitHub API로 브랜치 존재, 이슈 상태를 직접 확인합니다.

## 실제 도구 사용 과정

4단계 실행

1

**create\_repository**

새 레포지토리 생성



2

**initialize\_readme**

README 초기화



3

**create\_branch**

'dev' 브랜치 생성



4

**create\_issue**

이슈 #1 등록



실험 시 테스트용 GitHub 계정 필요 - 실제 레포에 영향을 줄 수 있음

성공률: 30.30% | 평균 스텝: 8.22



## 분석 목표

"Yahoo Finance에서 AAPL의 최근 6개월 주가 데이터를 가져와 이동평균(20일/50일)을 계산하고 골든크로스 여부를 판단하라."

주가 데이터

이동평균

골든크로스

## 핵심 포인트

숫자 정밀도가 중요한 도메인입니다. 요약/압축하면 정확도를 잃음. "SGD 83.00"을 "약 SGD 80대"로 요약하면 정답을 놓칩니다.

## 분석 과정

5단계 실행

1 Yahoo Finance 데이터 조회  
AAPL 6개월 주가 데이터 가져오기



2 20일 이동평균 계산  
 $MA(20) = (P1 + P2 + \dots + P20) / 20$



3 50일 이동평균 계산  
 $MA(50) = (P1 + P2 + \dots + P50) / 50$



4 골든크로스 판정  
 $MA(20) > MA(50)$  여부 확인



5 결과 보고  
분석 결과 및 시각화



정답 검증  
Evaluator가 실제 값과 비교



이 태스크는 숫자 정밀도가 중요하며, 시간 동기화가 필요합니다.

성공률: 67.50% | 평균 스템: 5.2



## 목표

"Blender를 사용하여 **3D 오브젝트**를 생성하고, **씬**을 구성한 후, **렌더링**하라. 오브젝트의 위치, 크기, 재질을 정확히 설정해야 한다."

오브젝트 생성

씬 구성

렌더링

## 핵심 포인트

이 태스크는 **순서 의존적**이며 **상태 관리**가 중요합니다. 오브젝트 생성 → 위치 설정 → 재질 적용 → 렌더링 순서를 따라야 합니다.

## 실제 도구 사용 과정

6단계 실행

1

### 3D 오브젝트 생성

기본 도형(큐브, 구체, 원통) 생성



2

### 위치/크기 설정

오브젝트의 좌표와 스케일 조정



3

### 재질/텍스처 적용

색상, 질감, 반사율 설정



4

### 씬 구성

카메라, 조명, 배경 설정



5

### 렌더링 설정

해상도, 샘플링, 출력 형식



6

### 결과 저장

이미지/동영상 파일 출력



이 태스크는 **순서 의존적**이며 **상태 관리**가 중요합니다.

성공률: 57.89% | 평균 스텝: 7.2



## 목표

Booking.com에서 **항공편을 검색**하고, "**항공편 없음**" 팝업이 등장했을 때 **검색 조건을 수정**하여 예상치 못한 상황에 대응하는 능력을 테스트합니다.

폼 작성

팝업 처리

조건 수정

## 핵심 포인트

이 태스크는 **동적 콘텐츠 스크래핑**과 **예외 처리** 능력을 테스트합니다. 예상치 못한 팝업 (항공편 없음 등)에 대한 대응 능력이 중요합니다.

## 실패 사례 분석

53.2% - Planning Error

**문제:** 항공편 없음 팝업이 등장했을 때, 에이전트가 **검색 조건을 수정하는 대신** 같은 페이지를 반복 스크래핑

**결과:** 스텝 예산 소진으로 실패

시도 횟수

5회

성공률

43.59%

## 올바른 접근

+7.69% 개선

**전략:** Exploration Phase(2단계 실행) - 탐색 단계에서 MCP 서버를 자유롭게 사용해 봄

**효과:** Browser Automation에서 성능 향상

**i** 이 태스크는 **웹 자동화**와 **예외 처리** 능력을 테스트합니다.

성공률: 43.59% | 평균 스텝: 6.54



## 목표

"최근 3년간 AI 모델의 파라미터 규모 변화에 대한 정보를 검색하고, 여러 소스에서 모은 데이터를 종합하여 보고하라."

정보 검색

소스 종합

증거 결합

## 핵심 도구



Google Search

웹 검색 엔진



Fetch MCP

웹 페이지 가져오기

## 멀티소스 검색 프로세스

4단계 실행

1

초기 검색

AI 모델 파라미터 규모 키워드 검색



2

소스 확장

관련 논문, 블로그, 뉴스 검색



3

정보 추출

각 소스에서 핵심 데이터 추출



4

종합 보고

여러 소스 정보를 하나로 정리



이 태스크는 여러 소스에서 정보를 수집하고 증거를 결합하는 능력을 테스트합니다.

성공률: 21.82% | 평균 스텝: 8.26



Part 3

## 실행 기반 Evaluator 설계

3가지 검증 방식  
Format + Static + Dynamic

### Format Evaluator

4개

에이전트 출력이 지정된 형식에 맞는지 검증합니다. JSON 형식, 필수 필드, 데이터 타입 등을 확인합니다.

- ✓ JSON 형식 검증
- ✓ 필수 필드 확인
- ✓ 데이터 타입 검증

예: {name, address, rating} 형식 검증

### Static Evaluator

32개

시간이 지나도 변하지 않는 고정 정답과 비교합니다. 미리 정의된 기준으로 PASS/FAIL을 판정합니다.

- ✓ 고정 정답 비교
- ✓ 정확도 측정
- ✓ 일관성 검증

예: GitHub 브랜치 존재 여부

### Dynamic Evaluator

48개

평가 시점에 실제 MCP 서버를 호출하여 실시간 정답을 확보한 뒤 비교합니다.

- ✓ 실시간 데이터 호출
- ✓ 동적 정답 비교
- ✓ 시간 민감성 검증

예: 현재 AAPL 주가 비교

### 평가 프로세스

3단계 검증

#### 1. 형식 검증

JSON/구조 확인



#### 2. 정적 검증

고정 정답 비교



#### 3. 동적 검증

실시간 데이터 비교

84

총 Evaluator

4

Format

32

Static

48

Dynamic



Part 3

# Dynamic Evaluator 사례

실시간 주가 검증  
AAPL 분석 예시

## 문제 상황

"현재 AAPL 주가를 알려줘" 시간 민감

이 태스크는 **시간에 민감한 정답**을 요구합니다. 1분마다 주가가 변하기 때문에, 평가 시점의 실제 주가와 비교해야 합니다.

Yahoo Finance

실시간 데이터

## 검증 프로세스

- Evaluator가 Yahoo Finance를 직접 호출
- 실제 주가와 모델 답변 비교
- 오차 범위 내에서 PASS/FAIL 결정

## 실제 실행 과정

3단계 검증

- 1

**쿼리 파라미터 고정**  
AAPL, 현재 시점, 1일 데이터
- 3

**결과 비교**  
실제 주가 vs 모델 답변



- 2

**Evaluator 호출**  
Yahoo Finance API 직접 조회
- 4

**PASS/FAIL 결정**  
±0.5% 오차 범위



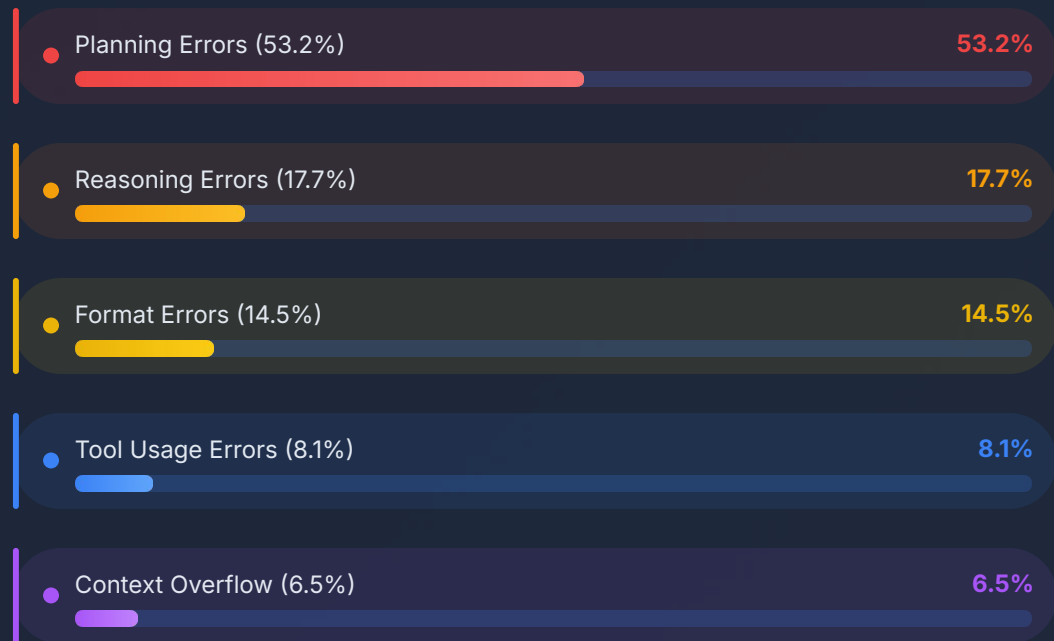
**i** Dynamic Evaluator는 시간에 민감한 태스크를 평가할 때 실시간 정답을 확보하여, **기존 벤치마크의 정적 정답 한계**를 극복합니다.

시간 민감성 대응





### 실패 유형별 분포



### 실패 유형별 상세 분석



⚠ Planning Errors가 53.2%로 가장 큰 비중을 차지합니다.

전체 실패의 절반 이상이 계획 관련 오류



## 사례 1: 항공편 팝업 대응 실패

**문제:** Booking.com에서 항공편을 검색하다 "**항공편 없음**" 팝업이 등장했을 때, 에이전트는 검색 조건을 수정하는 대신 **같은 페이지를 반복 스크래핑** 했습니다.

● — ● — ● 스텝 예산 소진 → 실패

✖ 실패 원인: 계획 수정 실패

에이전트가 예상치 못한 상황에서 계획을 수정하지 못했습니다.

## 사례 2: 숫자 환각

**문제:** Yahoo Finance에서 "**SGD 83.00**"을 정확히 추출했으나, 최종 답변에서 "**SGD 85.00**"으로 보고했습니다.



숫자 정밀도 문제

도구는 맞게 썼지만 답을 환각했습니다.



✖ 실패 원인: 추론 오류 (17.7%)

숫자 추출은 정확했으나 최종 보고에서 환각 수치 기입

## 실패 분석 통계

GPT-5 실패 케이스 수동 분석

53.2%

Planning  
계획 오류

17.7%

Reasoning  
추론 오류

14.5%

Format  
형식 오류

8.1%

Tool  
도구 사용

6.5%

Context  
컨텍스트 초과

💡 **핵심 인사이트:** Planning 오류가 53.2%로 가장 큰 비중을 차지합니다.

GPT-5 실패 케이스 수동 분석



Key Finding 1

# Long-Context Challenge

상호작용 턴 수  
<10턴 vs ≥10턴 성능 비교

## 성능 급락 현상

10턴 이상에서 성능 절반 이하로 감소



<10턴 **태스크**에서는 GPT-5가 **56.3%** 성공률을 보이지만, ≥10턴에서는 **24.8%**로 급락합니다.

## Claude 성능 비교



## 원인 분석

MCP 서버 응답 누적

서버 응답 누적  
매 턴마다 수만 토큰

컨텍스트 오버플로  
최대 컨텍스트 초과

시간 지연  
응답 처리 시간 증가

추론 저하  
복잡한 추론 실패



Key Finding 1

## Long-Context 완화: 요약 듀얼 에이전트

도메인별 효과 차이  
요약 전략의 한계

### 요약 듀얼 에이전트 구조

메인 에이전트 + 요약 에이전트를 두고, 각 스텝에서 서버 출력을 압축하여 context overflow 방지

#### 메인 에이전트

태스크 실행 및 결정

#### 요약 에이전트

정보 압축 및 요약

### 도메인별 효과 분석

- **Location Navigation:** 정보 밀도가 낮아 핵심(좌표, 거리) 추출 가능, 요약 효과적
- **Financial Analysis:** 압축 시 숫자 정밀도 손실 발생, 정답 누락 등 부정적 영향
- **Browser Automation:** 동적 웹 조작에서 요약이 효과적이지 않거나 오히려 성능 악화 초래

### 도메인별 요약 효과 비교

정밀도 vs 효율성

#### Location Navigation

개선

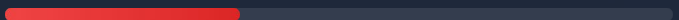
정보 밀도가 낮아 압축 용이



#### Financial Analysis

악화/미미

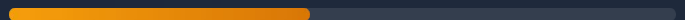
숫자 정밀도 손실로 부정적 영향



#### Browser Automation

불일치/악화

동적 콘텐츠 요약의 한계



💡 요약은 만능이 아니다 - 도메인의 정보 밀도에 따라 효과가 비대칭적으로 달라진다

MCP-Universe 분석



Key Finding 2

## Unknown-Tools Challenge

암묵적 제약 미인지  
도구 사용 패턴 학습 문제

### 문제 현상

#### ⚠️ 암묵적 제약

도구 설명에 명시되지 않은 제약 조건을 모름

#### ❓ 사용 패턴

정확한 파라미터 사용법을 알지 못함

**Yahoo Finance MCP**에서 주가 조회 시 **시작일=종료일**로 설정하면 API가 실패하는데, 도구 설명에 이 제약은 명시되어 있지 않습니다.

### 완화 전략: Exploration Phase

#### 🌐 Browser Automation +7.69%p

GPT-4.1 성능 향상 (→ 30.77%)

#### 📈 Financial Analysis +7.50%p

Claude-4.0-Sonnet 성능 향상 (→ 62.50%)

⚠️ 사전 탐색이 컨텍스트를 추가 소비하여, 도메인에 따라 오히려 성능을 저하시킬 수도 있음 (보편적이지 않음)

### 구체적 사례 분석

3가지 실패 유형

#### 📍 Location

좌표 계산 오류

두 호텔 중간 지점 계산 시 **좌표 정밀도** 문제로 실패

#### 🗄️ Repository

브랜치 생성 실패

**README 초기화** 옵션 누락으로 실패

#### 📈 Financial

데이터 정밀도

**숫자 추출**은 정확했으나 최종 보고에 **착각**

💡 Exploration Phase 전략: **사전 탐색(도구 사용법 파악)은 필수**로 일부 한계 완화

탐색이 컨텍스트를 추가 소비하는 부작용 존재



Key Finding 3

# Cross-Domain Variations

모델별 도메인 편차  
6개 도메인 성능 분석

## 모델별 도메인 성능 분포

성능 차이: 최대 40%+



## 도메인별 성능 패턴

상위 3개 모델



💡 핵심 인사이트: 모델마다 강한 도메인과 약한 도메인이 완전히 다르다 - GPT-5는 Finance에서 강하지만 Location에서 약함

DeepSeek-V3.1: Repo 0%

📌 도메인별 최적화가 필요하며, 특정 도메인 배포 시 해당 도메인 성적 확인 필요

특정 도메인 배포 시 해당 도메인 성적 확인 필요



## 추가 관찰: Tool Noise와 제품형 에이전트의 한계

Key Finding 4-5  
도구 선택 최적화

### Tool Noise: 무관한 도구 연결 시 성능 저하

Claude-4.0-Sonnet (Location Navigation)

22.22% → 11.11%

-11.11%p

GPT-4.1 (Browser Automation)

23.08% → 15.38%

-7.70%p

7개 MCP 서버(94개 도구)를 동시 연결했을 때 성능이 현저히 감소합니다. 무관한 도구의 존재가 모델의 판단을 교란시킵니다.

### 제품형 에이전트의 한계

리더보드 기준

26.41%

Cursor Agent

Claude-4.0-Sonnet-Thinking

30.30%

ReAct

Same Model

Cursor Agent가 범용 MCP 태스크에서는 기본 ReAct보다 낮은 성능을 보입니다 (코딩 특화의 한계).

### 산업계 대응: Tool Search (on-demand)

Anthropic 2025



수천 개 도구를 다 로딩하지 않고 필요한 것만 on-demand로 검색

MCP 정확도: 49% → 74%로 향상



+25%

정확도 향상



도구가 많을수록 올바른 도구 선택이 어려워짐 → 검색 기반 on-demand 권장

MCP-Universe 논문 및 산업 동향



Part 4

# 리더보드 : Overall Track 요약

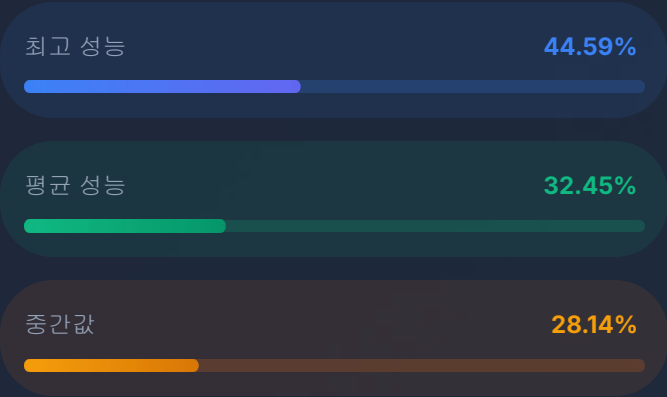
최고 성능

Gemini-3-Pro-Preview: 44.59%

## Top 10 모델 성능

| 순위 | 모델                                           | Overall SR    | 비고       |
|----|----------------------------------------------|---------------|----------|
| 1  | <b>Gemini-3-Pro-Preview</b><br>Function Call | <b>44.59%</b> | 최고       |
| 2  | <b>GPT-5-High</b><br>ReAct                   | <b>44.16%</b> | ReAct 최고 |
| 3  | <b>GPT-5-Medium</b><br>ReAct                 | <b>43.72%</b> | -        |
| 4  | <b>Grok-4.1-Fast</b><br>Function Call        | <b>40.69%</b> | -        |
| 5  | <b>Claude-4.5-Sonnet</b><br>Function Call    | <b>35.06%</b> | -        |

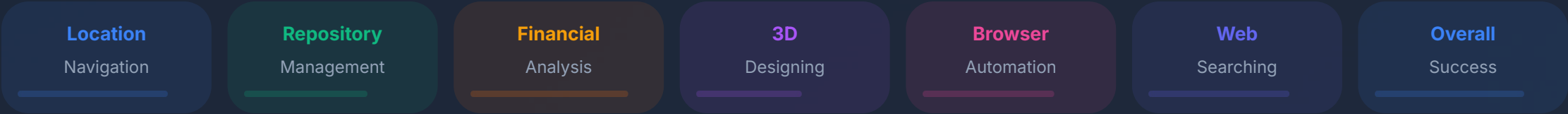
## 성능 분석







## 도메인별 성능 비교



## 모델별 성능

|                               |        |         |
|-------------------------------|--------|---------|
| <div></div> GPT-5-High        | 44.16% | Overall |
| <div></div> GPT-5-Medium      | 43.72% | Overall |
| <div></div> Grok-4            | 33.33% | Overall |
| <div></div> Claude-4.0-Sonnet | 29.44% | Overall |

## 도메인별 성공률



GPT-5 계열이 Financial Analysis에서 67.5%로 가장 높은 성능을 보임

최고 성능: 44.16%



Part 4

## 리더보드 : Function Call Track 비교

최고 성능  
Gemini-3-Pro-Preview: 44.59%

### Top 5 모델 성능

1 Gemini-3-Pro-Preview

44.59%

2 GPT-5-Medium

41.99%

3 Grok-4.1-Fast

40.69%

4 Claude-4.5-Sonnet

35.06%

5 Grok-4-Fast

32.47%

### 도메인별 성능 분석

Location

35.56%

Repository

18.18%

Financial

82.50%

3D Design

52.63%

44.59%

Gemini-3-Pro-Preview

최고 성능

41.99%

GPT-5-Medium

2위

40.69%

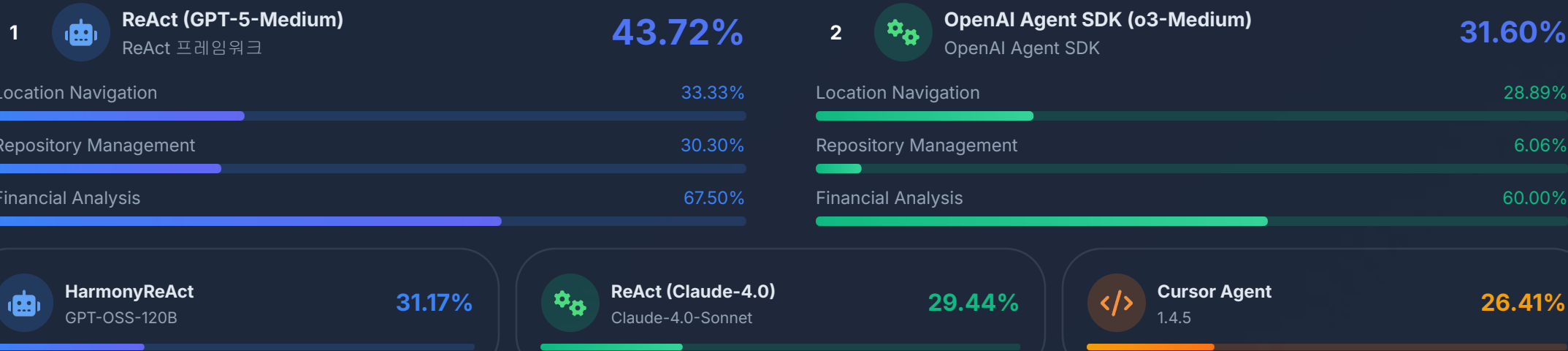
Grok-4.1-Fast

3위



## 에이전트 아키텍처별 성능

Overall Success Rate



ReAct (GPT-5-Medium)이 **43.72%**로 가장 높은 성능을 보임

총 8개 에이전트 평가



# 최신 동향 (2025~2026)

새로운 벤치마크  
보안 이슈 & 산업 동향

## 새로운 벤치마크

NEW

MCP-Universe 이후 등장한 새로운 벤치마크들

- MCP-Bench

Accenture, 다중 흡

28서버
- MCPAgentBench

외부 서비스 의존성 제거

시뮬레이션
- OSWorld-MCP

GUI + MCP 결합

ICLR'26

## 보안 이슈

ACTIVE

MCP 에이전트의 보안 취약성 분석

- GrantBox

권한 오용 평가

FSE 2026
- 프롬프트 인젝션

특히 LLM들이 취약

취약
- 권한 관리

세분화된 권한 설정

필요

## 산업 대응

UPCOMING

산업계의 MCP 표준화 및 도구 개선

- Tool Search

수천 개 도구 on-demand

Anthropic
- Agent Skills

도메인 전문성 패키징

표준화
- 관리형 MCP

원격 MCP 서버 지원

Google



## 결론: 벤치마크의 진화

핵심 메시지  
"폭 → 안정성 → 현실성"

### 벤치마크 진화의 3단계



### 벤치마크의 패러다임 전환

- ✓ 합성 데이터 중심에서 **실제 운영 API** 중심으로 이동
- ✓ 주관적 프롬프트 평가에서 객관적 **상태 검증**으로 진화
- ✓ 단일 도구 사용에서 **대규모 도구 공간** 및 **장기 추론** 테스트로 확장

### 벤치마크 진화의 핵심 특징

3가지 차원

**폭 (Breadth)**  
AgentBench

다중 환경에서의 폭넓은 평가. OS, DB, KG, 게임 등 8개 환경에서 29개 모델을 평가한 최초의 다중 환경 에이전트 벤치마크.

**안정성 (Stability)**  
StableToolBench

재현성 확보를 위한 가상 API 서버와 SoPR/SoWR 평가 방식. API 불안정성을 흡수하여 공정한 비교 가능.

**현실성 (Realism)**  
MCP-Universe

실제 MCP 서버에 직접 연결된 벤치마크. 11개 MCP 서버, 133개 도구, 231개 태스크로 구성.



결과적으로 에이전트 도구 사용 평가는 단순한 **폭 (Breadth)**과 **안정성 (Stability)**의 검증을 넘어, 이제 **현실성 (Realism)**을 갖춘 복잡한 실세계 상호작용의 단계로 진입했습니다.